27 DE OCTUBRE DE 2026

LABORATORIO DE COMPUTACIÓN

EDDY LIMA Y JOSÉ TROYA

UCOM418_81 SISTEMAS OPERATIVOS

2026

Contenido

Descripción_____ 2

Análisis_____ 2

Planificación y Diseño_____ 4

Implementación_____ 5

Conclusiones_____ 5

Bibliografía_____ 6

Descripción

El proyecto aborda la gestión automatizada de un laboratorio de computación académico, cuyo propósito principal es organizar, monitorear y administrar el uso de las computadoras disponibles por parte de los estudiantes. El problema central radica en la concurrencia: múltiples estudiantes (unidades de ejecución) intentan acceder, reservar y utilizar de forma simultánea un conjunto limitado de recursos compartidos (las estaciones de trabajo). Sin una correcta sincronización, se pueden presentar condiciones de carrera, asignaciones múltiples de una misma máquina o inconsistencias en el registro de disponibilidad y tiempos de utilización.

Análisis

En este escenario, las unidades de ejecución están representadas por los procesos de los estudiantes que solicitan una terminal, mientras que el recurso compartido principal corresponde al estado y disponibilidad de las computadoras del laboratorio, así como el registro centralizado de los tiempos de sesión. La cooperación y sincronización entre procesos es estrictamente necesaria para garantizar la exclusión mutua al momento de asignar una terminal libre, evitar el acceso concurrente indebido a los registros de tiempo y prevenir bloqueos mutuos (deadlocks) cuando la demanda de equipos supere la capacidad instalada.

A continuación, se presenta la tabla comparativa que detalla cómo se implementarían ambos modelos de concurrencia en la situación seleccionada:

Tabla 1 Datos del programa

Modelo	Mecanismo de Coordinación	Gestión del Recurso Compartido	Ventajas / Inconvenientes en el Escenario
Memoria Compartida	Se emplean estructuras de datos en memoria global (como arreglos de estado o tablas hash protegidas por semáforos y mutexes) donde se actualiza directamente el estado de cada computadora.	Los procesos de los estudiantes leen y modifican de forma directa las variables compartidas que representan la disponibilidad de las PCs y sus tiempos de uso.	Ventaja: Alta velocidad de acceso y actualización de estados en tiempo real. Inconveniente: Requiere un manejo riguroso de secciones críticas para evitar corrupción de datos por condiciones de carrera.
Paso de Mensajes	Los procesos se comunican enviando y recibiendo mensajes sincronizados a través de un proceso centralizador (servidor o gestor del laboratorio) mediante buzones (mailboxes) o colas de mensajes.	Un proceso administrador central mantiene el control exclusivo del estado de las computadoras y procesa las peticiones de asignación o liberación de manera secuencial o por canales seguros.	Ventaja: Elimina la necesidad de primitivas complejas de sincronización a nivel de memoria y facilita la escalabilidad distribuyendo solicitudes. Inconveniente: Mayor sobrecarga (overhead) por serialización, transmisión y enrutamiento de mensajes.

Planificación y Diseño

1. Identificación y Representación del Recurso Compartido

En la plataforma de gestión del laboratorio de computación, el recurso compartido principal está constituido por el inventario y estado operativo de las estaciones de trabajo, así como el registro de las asignaciones activas de los usuarios.

- Que representa: Representa la disponibilidad física y lógica de cada equipo en el laboratorio, vinculada al identificador del usuario asignado y al tiempo de inicio de su sesión.
- Por que requiere coordinación: Múltiples hilos solicitan una computadora libre al mismo tiempo. Sin una coordinación estricta, se producen condiciones de carrera donde dos usuarios reciben la asignación de la misma terminal o se corrompe el conteo global de equipos disponibles.

2. Formas de Representación del Recurso en Python

Para modelar el estado del recurso compartido en Python, se evalúan dos alternativas estructurales:

- Tipo Básico (Entero - Contador Global): Un número entero que almacena únicamente la cantidad total de computadoras disponibles en el laboratorio. Cada hilo decrementa o incrementa este contador al ocupar o liberar una máquina. Su limitación es que no permite identificar qué máquina específica está ocupada ni que usuario la utiliza.
- Tipo Complejo (Diccionario / Estructura anidada): Un diccionario de Python donde cada clave es el identificador de la computadora y su valor contiene un subdiccionario con atributos detallados: `{"pc_01": {"estado": "disponible", "usuario_id": None, "tiempo_inicio": 0}}`. Permite gestionar de forma granular la trazabilidad completa del laboratorio.

3. Diseño de Clases y Relación con los Hilos

Para dar soporte a la solución orientada a objetos en Python, se diseñan las clases:
Clase `LaboratorioManager` que es el gestor del recurso:

- Responsabilidad: Administrar la estructura de datos compleja del recurso compartido y proveer métodos seguros para consultar y modificar el estado de los equipos.
- Atributos: `estado_computadoras` es un diccionario complejo con el inventario y `lock` un objeto `threading.Lock` para exclusión mutua.
- Métodos: `asignar_computadora(usuario_id)` busca una PC libre y actualiza su estado, y `liberar_computadora(pc_id)` restablece el estado de la terminal a disponible.

Clase `UsuarioThread` la unidad de ejecución:

- Responsabilidad: Simular el comportamiento concurrente de un usuario interactuando con el sistema.
- Atributos: `id_usuario` y `gestor_lab` que referencia al `LaboratorioManager`.
- Métodos: `run()` que ejecuta el ciclo de vida del usuario solicitando acceso,
- simulando el tiempo de uso y liberando el equipo.

4. Identificación de la Sección Crítica y Mecanismo de Sincronización

La sección crítica del sistema comprende aquellas operaciones en las que se consulta el diccionario de equipos para localizar una terminal libre y se modifica de manera subsiguiente su estado de disponible a ocupado. Si múltiples hilos ejecutan estas acciones de forma simultánea, se produce una condición de carrera en la que dos o más usuarios podrán visualizar y reclamar la misma computadora al mismo tiempo, lo que generaría una asignación doble y la consiguiente corrupción o inconsistencia en los datos del registro.

Para prevenir este fallo y coordinar adecuadamente el acceso al recurso compartido bajo el modelo de memoria compartida, se implementará un cerrojo de exclusión mutua mediante un objeto `threading.Lock` en Python, garantizando de este modo que un solo hilo a la vez pueda ingresar a la sección crítica para verificar la disponibilidad y actualizar el estado de los equipos de forma segura. De igual forma, la operación de liberación (liberar_computadora) se ejecuta bajo el mismo Lock para mantener consistente el registro.

5. Comparación y Justificación de la Representación Seleccionada

Al evaluar las alternativas para modelar el estado del recurso compartido, se observa que la representación básica basada en un contador numérico entero ofrece gran ligereza y simplicidad en términos de implementación; sin embargo, resulta insuficiente para los requerimientos de la plataforma, ya que carece de la trazabilidad detallada indispensable para conocer qué usuario específico ocupa cada terminal y cuanto tiempo lleva en sesión. En contraste, la representación compleja mediante un diccionario estructurado proporciona un control absoluto y realista del laboratorio al almacenar metadatos clave por cada estación de trabajo. Por consiguiente, se ha decidido utilizar la representación compleja gestionada a través de la clase `LaboratorioManager`, debido a que satisface plenamente las necesidades funcionales del sistema al permitir un registro preciso de la disponibilidad, los tiempos de utilización y la titularidad de los equipos.

Implementación

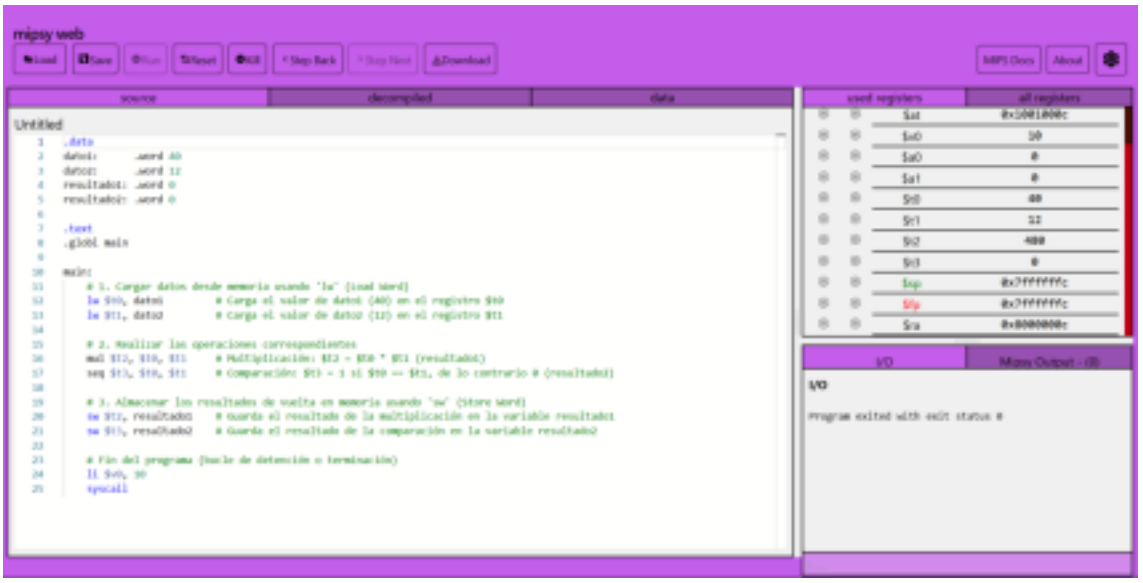


Figura 1 Captura de pantalla del programa

Conclusiones

A partir de la comparación realizada y evaluando las necesidades específicas del laboratorio de computación, se determina que el modelo de Memoria Compartida (respaldado por mecanismos de exclusión mutua como threading.Lock) resulta más adecuado para coordinar las unidades de ejecución. Dado que la disponibilidad de las computadoras y el registro de los tiempos de utilización requieren respuestas casi instantáneas y consultas concurrentes frecuentes por parte de los usuarios, el acceso directo a una estructura de datos centralizada minimiza la latencia y optimiza el rendimiento general del sistema académico.

Bibliografía

Citas APA – Normas APA. (n.d.). Retrieved September 24, 2026 from https://normas apa.org/citas/

CiteMaker CiteWeb | APA 7th Edn. - Chrome Web Store. (n.d.). Retrieved September 24, 2026 from https://chromewebstore.google.com/detail/citemaker-citeweb-apa7th/naankklphfojljboaokgfbheobbgenka

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Fundamentos de sistemas operativos (9ª ed.). McGraw-Hill.

GeeksforGeeks. (2025). Difference between shared memory and message passing. Recuperado de https://www.geeksforgeeks.org/difference-between-shared-memory-and-message passing/

Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). Operating Systems: Three Easy Pieces. Remzi Education. Disponible en: https://pages.cs.wisc.edu/~remzi/OSTEP/